

PWU_AB_4_APPLY_PORT

PWU-AB-4 — In-guest OTA agent ApplyPort: closing the loop to the Helix control plane

Revision: 2 **Last modified:** 2026-06-19T00:00:00Z **Status (§11.4.6):** DESIGN + SCAFFOLD — the ApplyPort Go interfaces/struct (step 2 of §8) and the healthy-marker shell script + systemd unit (step 3) are authored and compile-clean, but the persistent-env gating dependency (§3 / PWU-AB-2 §4.5) is UNVERIFIED and the full agent loop is NOT wired. See [docs/qa/20260619-pwu-ab4-scaffold/REPORT.md](#) for the scaffold capture evidence. Every “does not exist yet” / “UNVERIFIED” / “NOT wired” marker below is honest.

Authoritative parent design: [../.. /research/rk3588_emulator/REPORT.md](#) (PWU-AB-4 row, line 78; reuse decision lines 45/55/91). State machine the loop drives: [../.. /tests/emulator/ab_virt/u-boot_ab/README.md](#) (“A normal OTA apply → slot-switch”, lines 83–89; the explicit “healthy-boot reset is in-guest, NOT in boot.cmd” note, lines 76–80).

1. Scope and goal

PWU-AB-4 wires a real in-guest OTA agent that closes the OTA loop against the live Go control plane (server/), driving the **already-proven** U-Boot A/B mechanism end-to-end. The six steps of the apply sequence (the prompt’s a–f):

Step	Action	Mechanism
(a)	Receive an apply instruction over ota-protocol from the Go control plane	GET /api/v1/client/update → 200 UpdateAvailable (server/internal/api/handlers_client.go:20)
(b)	Write the new rootfs to the inactive slot	in-guest slot-writer (RAUC rauc install, or a dd-class writer for the no-verity path) → /dev/vda3 (B) when A active
(c)		

Step	Action	Mechanism
	Flip BOOT_ORDER + arm upgrade_available=1/ bootcount=1 in the U-Boot env	fw_setenv against the FAT boot partition env (does NOT exist yet — §3)
(d)	Reboot	reboot (guest) → U-Boot re-sources boot.scr (boot.cmd) → head slot = B
(e)	On a healthy boot, clear upgrade_available+bootcount	healthy-marker systemd unit / init script (does NOT exist yet — §4)
(f)	Report telemetry back	POST /api/v1/client/ telemetry lifecycle (server/internal/api/ handlers_client.go:105)

The headline: PWU-AB-1/AB-3 drive the U-Boot env **manually at the => prompt** in a single in-RAM session ([ab_rollback.sh:142–161](#) and its sibling [ab_slot_switch.sh](#)). PWU-AB-4 replaces that manual prompt-driving with a **guest agent that does steps (b)+(c)+(d) itself, persists the env across the power-cycle, and the healthy-marker (e) does the confirm** — exactly what the README §“A normal OTA apply” describes and exactly what a real RK3588/embedded U-Boot A/B target does.

2. The apply sequence mapped to REAL existing protocol + endpoints

Every wire type and endpoint named below **exists today** (cited file:line via codegraph). The agent reuses the canonical github.com/HelixDevelopment/ota-protocol types — it does NOT invent a parallel wire format (§11.4.27 no-fakes, §11.4.74 reuse-don’t-reimplement).

(a) Receive apply instruction — GET /client/update

- Handler: `Server.handleClientUpdate` — `server/internal/api/handlers_client.go:20`.
- Returns `204 No Content` when on-target, else `200` with `otaprotocol.UpdateAvailable` (`handlers_client.go:69–100`).
- Wire type: `otaprotocol.UpdateAvailable` — `submodules/ota-protocol/types.go:87`. Carries `ReleaseID`, `DeploymentID` (self-served so telemetry needs no out-of-band id — `types.go:95`, `handlers_client.go:74`), `Version`, `URL` (Range-served `.zip`, `handlers_client.go:76 / artifactURL :229`), `Offset`, `Size`, `SHA256`, `Signature`, `PayloadProperties`, optional `Delta`.
- On-target compare uses `otavalidator.CompareDotted` (`handlers_client.go:56`) — the agent’s `current_version` query param (`?current_version=...`, the Tier-1 emulator already sends it: `server/internal/deviceemu/emulator.go:247–249`) decides `200` vs `204`.

The agent must first **register** to obtain a device-scoped bearer token: `POST /devices/register` → `DeviceRegistrationResponse` (`ota-protocol/types.go:68`), exactly as the Tier-1 emulator does (`deviceemu/emulator.go:182-232`). Operator login (`POST /auth/login`) precedes registration (`emulator.go:155-176`).

(b) Write new rootfs to inactive slot

This is the step the Tier-1 Go emulator **fakes** (its package doc is explicit: “The only thing it fakes is the act of flashing — instead of writing a slot it advances its in-memory `CurrentVersion`”, `deviceemu/emulator.go:9-12`, `ApplyAndReport` :281-311). PWU-AB-4 makes it REAL inside the guest:

- Download the payload from `UpdateAvailable.URL` (Range-served identity-encoded `ZIP_STORED`) to a local path.
- Verify SHA256 + signature BEFORE writing (the agent’s safety-critical gate — mirrors the Android agent’s verify-before-apply: `OtaPollWorker.runCycle` step 3, `submodules/ota-android-agent/.../poll/OtaPollWorker.kt:76-94`). A poisoned artifact NEVER reaches the slot-writer.
- Write the verified payload to the **inactive** slot partition. The active slot is known from `helix_slot=<A|B>` on `/proc/cmdline` (set by `boot.cmd:100`) and `/etc/slot_id` (`assemble_ab_disk.sh:177-200`); inactive = the other. Slot→partition map is the load-bearing GPT contract: `A=/dev/vda2`, `B=/dev/vda3` (`uboot_ab/README.md:96-100`, `boot.cmd:34-40`).
- **Reuse decision for the writer (§11.4.74):** RAUC is already in the Buildroot config (`build_image.sh:115` `BR2_PACKAGE_RAUC=y`) and is the REPORT’s chosen in-guest A/B client (REPORT lines 43/87). The slot-writer SHOULD be `rauc install <bundle>` for the dm-verity path (PWU-AB-2), which writes the inactive slot AND owns the slot-state. For the pre-verity no-RAUC-bundle path a `dd/debugfs` writer mirroring `assemble_ab_disk.sh:204-205` is the fallback. **Neither writer is wired yet.**

(c) Flip BOOT_ORDER + arm the env — fw_setenv (§3 — does NOT exist yet)

After a successful inactive-slot write, the agent arms the new slot exactly as `uboot_ab/README.md:84-86` specifies:

```
BOOT_ORDER="B A"    upgrade_available=1    bootcount=1    (then persist)
```

Today this is done by hand at the `=>` prompt (`ab_rollback.sh:145-152`). PWU-AB-4 must do it from inside the guest via `fw_setenv` writing the SAME env the bootloader reads — see §3.

(d) Reboot

reboot in the guest. On the next power-on U-Boot re-sources `boot.scr` (`boot.cmd`), which: applies lazy defaults only if env empty (`boot.cmd:45-48` — so a persisted env survives), runs the bootcount guard (`boot.cmd:59-69`), selects head

B, boots root=/dev/vda3 helix_slot=B (boot.cmd:75-102). Because upgrade_available=1, U-Boot increments bootcount each reboot (README:44) — the probation window the healthy-marker must close.

(e) Healthy-boot confirm — clear upgrade_available+bootcount (§4 — does NOT exist yet)

The U-Boot doc and uboot_ab/README.md:76-80 are explicit: the healthy reset is “the responsibility of some application code (typically a Linux application)” and is intentionally NOT in boot.cmd. PWU-AB-4 supplies that application code — see §4. On a confirmed-healthy boot it runs fw_setenv upgrade_available 0; fw_setenv bootcount 0, freezing the counter ⇒ slot CONFIRMED good ⇒ no rollback. If the marker is never reached (unhealthy slot), bootcount climbs past bootlimit and the proven PWU-AB-3 guard swaps BOOT_ORDER back ⇒ auto-rollback to the previous-good slot.

(f) Report telemetry — POST /client/telemetry

- Handler: Server.handleClientTelemetry — server/internal/api/handlers_client.go:105. Returns 202 TelemetryAck.
 - Wire type: otaprotocol.TelemetryReport (ota-protocol/types.go:150) batched on the server-side api.TelemetryReport (server/internal/api/wire.go:207), which carries an optional Health block: api.TelemetryHealth{ BatteryPct, StorageFreeMB, ActiveSlot } (wire.go:200-205).
 - Events: the exactly-six otaprotocol.TelemetryEvent (ota-protocol/enums.go:200-207): download_started → installing → installed → verifying → success, or failure. The Tier-1 emulator already drives this exact lifecycle (deviceemu/emulator.go:285-291).
 - **deployment_id is REQUIRED by the schema validator** (handlers_client.go:140-159 via otatelemetry.NewEvent(...).Validate()); the agent self-serves it from UpdateAvailable.DeploymentID (no out-of-band step — types.go:95, emulator resolveDeploymentID :337-344).
 - **active_slot in the health block is the PWU-AB-4 value-add:** the server already persists it (applyDeviceRuntime handlers_client.go:220-222 → store.Device.ActiveSlot store/store.go:50 → postgres.go:122). A REAL guest can report the slot it actually booted (helix_slot from /proc/cmdline, /etc/slot_id), where the Tier-1 emulator has no real slot to report. This is the honest end-to-end signal that proves the loop closed on a real slot switch.
-

3. The fw_setenv/fw_printenv + /etc/fw_env.config requirement (NEW — shared with PWU-AB-2)

This is the single biggest not-yet-built piece and the load-bearing dependency for steps (c) and (e).

Why it is needed

`boot.cmd/ab_rollback.sh` today set the env at the interactive U-Boot prompt in a single in-RAM session; the `saveenv` in the rollback guard (`boot.cmd:68`) is the ONLY persistence and PWU-AB-1/AB-3 explicitly note the “persistent `saveenv`-across-power-cycle path is a later PWU” (`ab_rollback.sh:48–49`). For the agent to arm a slot (c) and the marker to confirm it (e), the guest userspace must read AND write the **same persistent U-Boot environment** the bootloader consults. The canonical Linux tool for this is `fw_setenv/fw_printenv` from `u-boot-tools` (`libubootenv`), driven by a config file `/etc/fw_env.config` that tells the tool WHERE the env blob lives.

What must be built (honest: none of this exists)

1. **A persistent U-Boot env blob the bootloader reads.** `boot.cmd` currently relies on lazy defaults + an in-session env; there is no `mkenvimage` blob placed on the disk yet (the README:17 notes the binary env blob is “later PWU”). PWU-AB-4 (or a precursor) MUST: (i) generate a binary default env from `uboot_ab/uboot.env` via `mkenvimage`, (ii) place it at a fixed offset/partition the QEMU U-Boot is configured to use (`CONFIG_ENV_IS_IN_*`), and (iii) confirm the same `boot.cmd` reads it (the lazy-default guards already tolerate a populated env, `boot.cmd:45–48`).
2. **`/etc/fw_env.config` in BOTH slot rootfs images** pointing at that exact env location (device path + offset + size + sector size). It MUST match the U-Boot env storage config byte-for-byte — a mismatch means the guest writes an env the bootloader never reads (a silent §11.4.108 SOURCE→RUNTIME gap, exactly the class the constitution warns about). This is added in `build_image.sh` (Buildroot rootfs overlay) + verified by `assemble_ab_disk.sh`.
3. **`u-boot-tools (fw_setenv/fw_printenv)` in the guest** — a Buildroot package add (`BR2_PACKAGE_UBOOT_TOOLS` or equivalent) in `build_image.sh`.

PWU-AB-2 research findings: `CONFIG_ENV_IS_IN_FAT` vs raw env region

The PWU-AB-2 design doc (`PWU_AB_2_RAUC_VERITY.md §4.5`) documents the critical finding about the QEMU `qemu_arm64` U-Boot defconfig: it **most commonly uses `CONFIG_ENV_IS_IN_FAT`** — the environment is stored as a **file** (`uboot.env`) on the FAT boot partition, NOT at a raw offset on the boot device. This has three consequences for the shared `fw_env` mechanism:

1. **Classic raw `fw_env.config` form does NOT directly address a FAT-file env.** The `<device> <offset> <size>` raw form (`/dev/vda1 0x000000 0x4000 0x200`) works only for a raw env region at a fixed offset. For a FAT-file env, either (A) recent `libubootenv` supports a FAT-file entry format, or (B) the U-Boot config is changed to `CONFIG_ENV_IS_IN_* raw` — placing the env at a fixed offset on the boot device — which the raw `fw_env.config` line then matches.

2. **The `build_image.sh fw_env.config` is COMMENTED OUT** as of the proven 2026-06-19 build. The `rootfs-overlay` at `build_image.sh:111-115` emits a commented `fw_env.config` with the explicit note: `# UNVERIFIED – guest has no MTD subsystem (proven 2026-06-19)`. Commented out until the `env` mechanism is resolved.
3. **Resolution path (RECOMMENDED by the PWU-AB-2 doc):** wire a **raw `env` region** on the boot device so that the classic `fw_env.config` raw form works — this is the simplest form for `fw_setenv/fw_printenv` and is platform-agnostic. The alternative (FAT-file `env` via `libubootenv`) depends on the build-time `u-boot.bin` configuration and may not be accessible from guest userspace without additional tooling. See PWU-AB-2 doc §4.5 for both candidate forms and the §11.4.108 verification warning.

The `/etc/fw_env.config` file is authored at `tests/emulator/ab_virt/rauc/fw_env.config` (and its `rootfs-overlay` sibling at `tests/emulator/ab_virt/rootfs-overlay/etc/fw_env.config`). Both carry the same UNVERIFIED annotation and the same two-path documentation. A future PWU step (the bundle-build PWU) uncomments and corrects the numbers after reading the built `u-boot.bin` config and round-trip-verifying against U-Boot's actual `env` storage.

Shared with PWU-AB-2 (RAUC dm-verity)

RAUC's U-Boot integration uses the **same** `fw_setenv/fw_printenv` + `BOOT_ORDER/upgrade_available` `env` to drive slot state (RAUC U-Boot bootchooser; REPORT line 43). So building `/etc/fw_env.config` + the persistent `env` blob is a SHARED prerequisite of PWU-AB-2 and PWU-AB-4 — build it once, both consume it. RAUC's `rauc-mark-good/bootchooser` is in fact a candidate to BE the healthy-marker of §4 (reuse over hand-rolled).

4. The healthy-marker mechanism (SCAFFOLDED)

The marker is the in-guest application code the U-Boot doc/README:76-80 require: on a confirmed-healthy boot it clears `upgrade_available+bootcount`, freezing the counter so the just-applied slot is CONFIRMED and no rollback fires.

Design (systemd unit OR init script — the guest is Buildroot)

The Buildroot guest's init system determines the form (Buildroot defaults to BusyBox init unless systemd is selected; `build_image.sh` config decides). Two equivalent shapes:

- **systemd unit** `helix-ab-confirm.service` — `Type=oneshot`, `After=multi-user.target` / ordered after the services whose liveness defines “healthy”, `ExecStart=/usr/bin/helix-ab-confirm`.
- **BusyBox init script** `/etc/init.d/S99helix-ab-confirm` run last in the boot sequence.

Either invokes a small `helix-ab-confirm` program that:

1. Confirms the system is genuinely healthy (NOT merely “init reached the marker”). The honest health predicate (§11.4.6 — never assume) **MUST** be a real check, e.g. the OTA agent registered + polled the control plane successfully (closing the loop is itself the health signal), key services up, `/etc/slot_id` matches the `helix_slot` the bootloader selected.
2. **ONLY** on a true healthy verdict: `fw_setenv upgrade_available 0` and `fw_setenv bootcount 0` (via §3’s tooling).
3. Is idempotent + crash-safe — re-running it on an already-confirmed slot is a no-op (read `upgrade_available` first; if already 0, do nothing).

Reuse note (§11.4.74)

RAUC ships `rauc status mark-good` (and its U-Boot bootchooser integration writes exactly the `upgrade_available/BOOT_ORDER` env). Since RAUC is already in the guest (`build_image.sh:115`), the healthy-marker **SHOULD** reuse `rauc ... mark-good` rather than a bespoke `fw_setenv` script wherever the RAUC bundle path (PWU-AB-2) is used. The bespoke `helix-ab-confirm` is the fallback for the pre-RAUC-bundle no-verity path and the place to anchor the loop-closed health predicate.

5. Reuse-vs-new decision (§11.4.74) — does `cmd/ota-device-emu` drive the real guest, or is a thin in-guest agent needed?

Decision: a NEW thin in-guest agent is needed. `cmd/ota-device-emu`’s apply logic does NOT extend to drive the real guest — but its protocol client is the reuse seam.

Rationale, grounded in the code:

- `deviceemu.Device` (`server/internal/deviceemu/emulator.go`) is a faithful, **REAL protocol** client: login, register, check-update, telemetry are all real HTTP against the canonical `ota-protocol` types (package doc :1–26). That half is exactly what the guest agent needs and **SHOULD** be reused conceptually (and the Go binary can even run **INSIDE** the guest if a Go runtime is present, or be cross-compiled `aarch64-static`).
- **BUT** `deviceemu` deliberately **fakes the flash**: `ApplyAndReport` advances an in-memory current string instead of writing a slot (:9–12, :305–309). It has **NO** concept of the inactive slot, **NO** `fw_setenv`, **NO** reboot, **NO** healthy-marker. Steps (b), (c), (d), (e) are precisely the faked/absent part. Extending `deviceemu` to do real slot writes + env flips + reboot would entangle the host-side Tier-1 emulator with guest-only OS operations it must **NOT** have — a decoupling violation in the wrong direction.
- The Android agent (`submodules/ota-android-agent`) already establishes the **RIGHT** shape: a thin worker (`OtaPollWorker.runCycle`, .../`poll/OtaPollWorker.kt:59–107`) that orchestrates `poll→download→verify→apply` over decoupled ports, with the OS-apply hidden behind `ApplyPort` (.../`apply/ApplyPort.kt:35–38`). For Android the impl is

ReflectiveUpdateEngineApplyPort (drives android.os.UpdateEngine, .../apply/ReflectiveUpdateEngineApplyPort.kt). **The A/B-virt guest is a Linux/U-Boot target with NO update_engine** — so it needs a DIFFERENT ApplyPort implementation: a UBootSlotApplyPort that does (b) inactive-slot write + (c) fw_setenv arm, leaving the reboot to the worker (mirroring how the Android bridge’s rebootToNewSlot is a separate call, UpdateEngineBridge.kt:60).

Concrete recommendation:

Piece	Reuse / New	Where it lives
Protocol client (login/register/check-update/telemetry over ota-protocol)	REUSE the deviceemu/ota-android-agent contract (same wire types)	guest agent reuses ota-protocol; MAY embed a cross-compiled deviceemu-style Go client
ApplyPort interface (decoupling seam)	REUSE the established ApplyPort concept (§11.4.28)	conceptually the same seam as ota-android-agent ApplyPort.kt:35
UBootSlotApplyPort impl (write inactive slot + fw_setenv arm)	NEW (the Linux/U-Boot analogue of ReflectiveUpdateEngineApplyPort)	helix_ota glue, in-guest (REPORT line 55 “agent ApplyPort→in-guest slot-writer glue stays in helix_ota”)
Inactive-slot writer	REUSE rauc install (verity path) / dd-class (no-verity fallback)	guest (RAUC already in image)
Persistent U-Boot env + /etc/fw_env.config + u-boot-tools	NEW (§3) — SHARED with PWU-AB-2	build_image.sh + assemble_ab_disk.sh
Healthy-marker	NEW (§4) , REUSE rauc mark-good where applicable	guest systemd unit / init script

This matches REPORT lines 45/55/91: reuse RAUC+U-Boot (in-guest) + the OTA glue (ApplyPort/protocol/server) stays in helix_ota; the server is UNCHANGED (the emulator registers/polls /api/v1 like a real board). No new ota-protocol message is required (see §6).

6. ota-protocol message gap analysis (§11.4.6 — do NOT invent APIs)

The existing ota-protocol surface is **sufficient** for the full PWU-AB-4 loop. No new message type is required:

- 1. covered by UpdateAvailable (types.go:87) + DeviceRegistrationRequest/Response (types.go:56/68) + UpdateCheckRequest (types.go:80).

- 1. covered by TelemetryReport/TelemetryEvent (types.go:150, enums.go:200) + the server-side TelemetryHealth.ActiveSlot block (wire.go:200–205).
- The slot/env/reboot/marker steps (b)–(e) are GUEST-LOCAL operations with NO wire representation by design — the control plane is intentionally unaware of slots (the device owns the A/B mechanism; only the resulting active_slot + lifecycle is reported back). This is correct and matches the existing applyDeviceRuntime slot handling (handlers_client.go:220–222).

One honest optional refinement (NOT required, flagged not assumed): the existing TelemetryEvent set has no distinct “booted-pending-confirm” vs “confirmed-good” event — the Android agent models this richer state internally (AgentState.BOOTED_NOT_YET_SUCCESSFUL / SUCCESSFUL, TelemetryReporter.kt:18–19) but the wire success event is the single terminal. For PWU-AB-4 the success event reported AFTER the healthy-marker fires (e) is the honest “confirmed-good” signal; reporting installed/verifying BEFORE reboot and success only AFTER the post-reboot marker correctly maps the U-Boot probation window onto the existing six events. If a future PWU wants the control plane to SEE the probation window explicitly, that would be a proposed additive TelemetryEvent (e.g. booted_pending_confirm) — **proposed, not built, and out of PWU-AB-4 scope.**

7. Test plan (§11.4.115 RED→GREEN + §11.4.5/§11.4.69 captured evidence)

Feature class (§11.4.69): boot_service. Evidence root: docs/qa/<run-id>-ab-apply-loop/. The harness reuses the proven expect-driver + QEMU-virt+HVF pattern of ab_rollback.sh/ab_slot_switch.sh (single Tcl quoting level, login-with-retry, \\$(...) guest-shell substitution), driven against a LIVE control-plane instance (the Go server + the Tier-1 contract the emulator already exercises).

RED→GREEN polarity (§11.4.115 — reproduce on the broken artifact, one polarity switch)

A single test source with RED_MODE (default 1):

- **RED (RED_MODE=1, pre-agent artifact):** boot the guest on slot A; trigger an apply via the control plane (publish a release/deployment so GET /client/update returns 200). With NO agent wired, assert the loop does NOT close: post-reboot helix_slot/etc/slot_id is STILL A, AND the server’s active_slot for the device is unchanged. This captures the defect-present baseline (the loop is open).
- **GREEN (RED_MODE=0, agent-wired artifact):** same stimulus; assert post-reboot the guest booted slot B (/proc/cmdline helix_slot=B, findmnt / = /dev/vda3, /etc/slot_id=B), the persistent U-Boot env shows BOOT_ORDER="B A" then (after marker) upgrade_available=0/bootcount=0 (via fw_printenv), AND the control plane received the success telemetry with active_slot=B (assert via the server’s device record / telemetry store).

Captured-evidence battery (§11.4.5/§11.4.69/§11.4.107 — never a single frame, never metadata-only)

Per run, under docs/qa/<run-id>-ab-apply-loop/:

1. `console.log` — full U-Boot + kernel + guest console (the expect `log_file`), showing the `boot.cmd` selection line (`boot.cmd:88`) for the NEW slot.
2. `fw_env_before.txt` / `fw_env_after.txt` — `fw_printenv` of `BOOT_ORDER/upgrade_available/bootcount` before arm, after arm, after healthy-marker (the env transition is the §11.4.108 runtime signature for steps c+e).
3. `slot_block_hash.txt` — pre/post block hash of the inactive-slot partition proving REAL bytes were written (step b is not faked — contrast with `deviceemu` which has no slot).
4. `server_device_record.json` — the control-plane device record showing `active_slot=B` + `current_version=new` (proves f closed the loop, captured from the live server, §11.4.13 downstream/sink-side evidence).
5. `telemetry.jsonl` — the accepted lifecycle events (`download_started...success`) with the resolved `deployment_id` (proves the 202 accepted, `rejected=0`).
6. `verdict.txt` — PASS/FAIL with the contrast summary.

Liveness (§11.4.107): a post-login sentinel (`HELIX_DONE..._MARK`) proves the guest shell is live, never a frozen frame — same technique as `ab_rollback.sh:203`.

Composed proofs (full test-type coverage, §11.4.27)

- **Healthy path:** `apply`→`write B`→`arm`→`reboot`→marker confirms→success w/ `active_slot=B`; `upgrade_available` ends 0.
- **Unhealthy path (composes proven PWU-AB-3):** `apply`→`write B`→`arm`→`reboot`→marker NEVER fires (inject an unhealthy slot B) → `bootcount` climbs past `bootlimit` → `boot.cmd` guard swaps `BOOT_ORDER` → auto-rollback to A → agent reports failure/rollback. This is the PWU-AB-3 rollback now TRIGGERED by a real agent-armed probation window rather than a hand-set `bootcount`.
- **Chaos/stress (§11.4.85):** kill the agent mid-slot-write (resumable, no brick); power-cut during the `fw_setenv` arm via QMP (the env is either old-and-consistent or new-and-consistent — never half-written-and-bricked); N-iteration determinism (§11.4.50) — identical PASS + identical evidence hashes across N runs on the same disk image MD5.
- **§1.1 paired mutation:** strip the healthy-marker's `fw_setenv upgrade_available 0` → the GREEN test MUST FAIL (the slot never confirms, so a later reboot rolls back when it should not) — proving the marker assertion is real, not a bluff. Second mutation: make `UBootSlotApplyPort` skip the inactive-slot write but still arm the env → `slot_block_hash` unchanged → test MUST FAIL.

§11.4.116 sync channel + §11.4.83 transcript

The harness emits a JSONL verdict stream + atomically-rewritten status snapshot (each verdict carries its evidence path), and the full bidirectional console transcript is the `docs/qa/<run-id>/` proof artifact.

8. Build order (honest dependency chain)

1. **PREREQUISITE (shared with PWU-AB-2):** persistent U-Boot env blob + /etc/fw_env.config + u-boot-tools in the guest (§3). Until this lands, the agent CANNOT persist a slot arm and the marker CANNOT confirm. This is the gating piece. **UNVERIFIED** — see §3 **CONFIG_ENV_IS_IN_FAT discussion**.
2. UBootSlotApplyPort (new in-guest ApplyPort impl, §5) + inactive-slot writer (RAUC or dd fallback). **SCAFFOLDED** — interfaces/struct in server/internal/device/applyport.go, fw_setenv backend in server/internal/device/fwenv.go; the SlotWriter interface is defined but has no concrete implementation wired yet.
3. Healthy-marker unit/script (§4). **SCAFFOLDED** — tests/emulator/ab_virt/in-guest/helix-ab-confirm.sh (bash, sh -n clean) + helix-ab-confirm.service (systemd unit). Not yet deployed into the guest image.
4. Guest agent driver tying poll→download→verify→apply→arm→reboot, reusing the ota-protocol client contract. **DESIGN ONLY** — no code exists for this step.
5. The RED→GREEN apply-loop test (§7) against a live server. **DESIGN ONLY** — the test plan is written; no test script exists.

Every step lands with four-layer coverage (§11.4.4(b)) + independent review (§11.4.125/§11.4.142) + §1.1 mutation, evidence under docs/qa/<run-id>/ (§11.4.83), per the REPORT's per-PWU contract (line 81).

Sources verified 2026-06-11; scaffold evidence 2026-06-19

- Scaffold capture: docs/qa/20260619-pwu-ab4-scaffold/REPORT.md (go build ./server/internal/device/ clean, sh -n on the healthy-marker script clean, source lines counted).
- In-repo code (cited file:line throughout): submodules/ota-protocol/types.go, enums.go; server/internal/api/handlers_client.go, wire.go; server/internal/deviceemu/emulator.go; server/cmd/ota-device-emu/main.go; submodules/ota-android-agent/.../poll/OtaPollWorker.kt, .../apply/ApplyPort.kt, .../apply/ReflectiveUpdateEngineApplyPort.kt; tests/emulator/ab_virt/uboot_ab/{README.md, boot.cmd}, ab_rollback.sh, assemble_ab_disk.sh, build_image.sh; docs/research/rk3588_emulator/REPORT.md.
- U-Boot Boot Count Limit (bootcount/bootlimit/altbootcmd/upgrade_available; healthy-reset is application responsibility): <https://docs.u-boot.org/en/latest/api/bootcount.html>
- U-Boot environment tools fw_setenv/fw_printenv + fw_env.config: <https://docs.u-boot.org/en/latest/develop/environment.html> (libubootenv u-boot-tools).

- Mender/RAUC U-Boot integration (BOOT_ORDER, swap-the-rootfs altbootcmd, fw_setenv-driven slot state): <https://docs.mender.io/operating-system-updates-yocto-project/board-integration/bootloader-support/u-boot/manual-u-boot-integration>
- RAUC A/B + QEMU + format=verity + U-Boot bootchooser / mark-good: <https://rauc.readthedocs.io/en/latest/examples.html>

Negative finding (§11.4.99): RAUC's official full-system QEMU example is x86+GRUB, not arm64+U-Boot; the arm64+U-Boot `fw_env.config/env-storage` offset is target-specific and MUST be verified against THIS emulator's QEMU U-Boot `CONFIG_ENV_IS_IN_*` once built (it does not exist yet) — do not assume the upstream example's offsets transfer.